

Laboratorio: Deep Learning Desde Cero con CNNs

En este laboratorio ganarán experiencia práctica con armar, entrenar y evaluar una red neuronal profunda. En particular armaremos una red convolucional (CNN por “Convolutional Neural Network”) para clasificar imágenes utilizando Python3 y la librería Keras. Los objetivos del laboratorio son:

- entender la estructura y el propósito de una red profunda,
- cargar y procesar un dataset,
- armar una red convolucional utilizando una librería estándar,
- elegir una métrica de pérdida (“loss”) y un optimizador apropiados,
- entrenar el modelo de distintas formas y analizar los resultados,
- evaluar la performance utilizando métricas adecuadas,

Deberían encontrar junto a éste pdf un notebook que pueden correr en Google Colab, seleccionando el runtime con GPU (“Change Runtime Type” > “T4 GPU” o similar). Les dejamos adicionalmente una lista de referencias para tener en cuenta mientras siguen el laboratorio, por si les surgen dudas.

Referencias:

- [Adam: A Method for Stochastic Optimization](#): el método de optimización estocástico más popular, y el paper más citado de la última década.
- [Keras: The high-level API for TensorFlow](#): documentación de Keras, la API de tensorflow que utilizaremos para armar y entrenar nuestras redes profundas. Alternativamente se puede utilizar [PyTorch](#), cuya sintaxis es similar.
- [Tensorboard: TensorFlow's visualization toolkit](#): interfaz gráfica para visualización de métricas (y más) en tiempo real de las redes (opcional). Se puede usar de forma manual con PyTorch, pero en Tensorflow los modelos loggean ciertos datos automáticamente.
- [Gradient-based learning applied to document recognition \(link alternativo\)](#): primer paper que utiliza CNN para reconocimiento de dígitos numéricos (MNIST).
- [CS231n Convolutional Neural Networks \(CNNs / ConvNets\)](#): página web del curso de deep learning de la Universidad de Stanford explicando CNNs

Introducción

Nos proponemos realizar un sistema que permita digitalizar documentos de texto manuscritos. En particular nos interesa la lectura automática de direcciones de correo postal, cheques y formularios.

Para lograr nuestro objetivo nos es necesario un sistema que pueda mirar letras y dígitos, y los clasifique correctamente en sus categorías. Para ésto es claro que necesitamos armar un sistema de **clasificación de imágenes**, que es precisamente para lo que sirven las redes neuronales convolucionales (CNN).

Asumiremos ciertas propiedades deseables sobre las imágenes que entran a nuestro sistema:

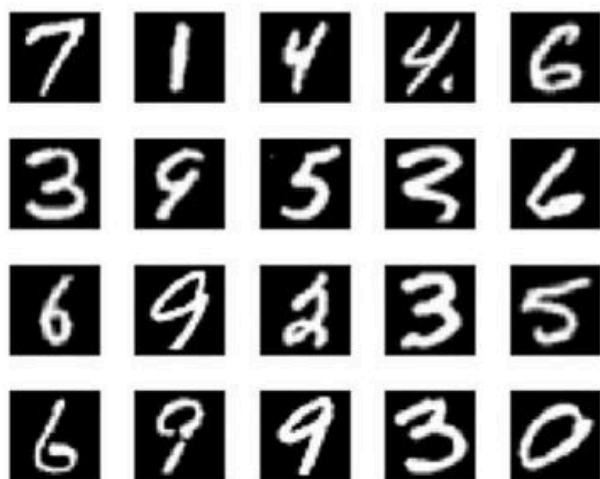
- cada imagen contiene un único caracter,
- los caracteres vienen pre-segmentados,
- el trazo del caracter se encuentra completamente contenido en la imagen,
- etc...

en verdad, asumiremos que nuestras entradas se asemejan a las de los datasets que mencionaremos a continuación (caso contrario, deberíamos conseguirnos un dataset propio).

Laboratorio: Deep Learning Desde Cero con CNNs

Parte 1 - Dataset

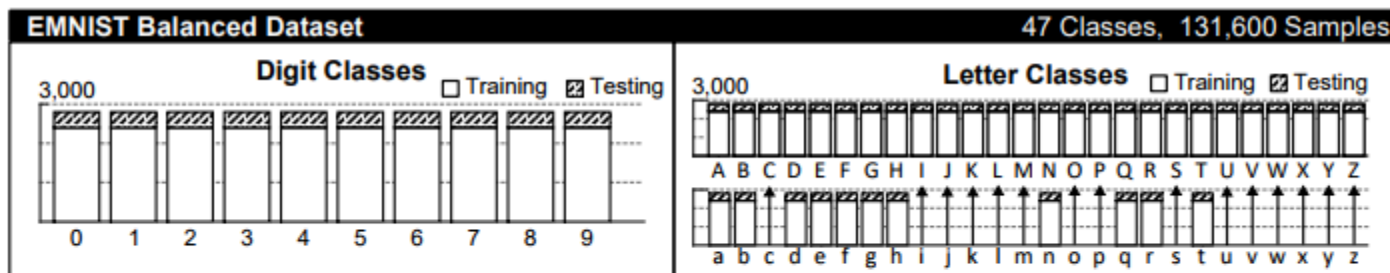
Utilizaremos dos datasets bastante conocidos, llamados MNIST (por “Modified National Institute of Standards and Technology database”) y EMNIST (por “Extended MNIST”), que se generaron por el instituto de estándares estadounidense y que se usan extensivamente para testear nuevas arquitecturas de aprendizaje automático. El segundo contiene y extiende al primero. En nuestro caso funcionan muy bien para entrenar una red relativamente sencilla.



Como podemos ver, los caracteres (dígitos numéricos y letras) están recortados individualmente, tienen fondo negro y trazo blanco, y ocupan casi el total de la imagen. Podemos ver que algunas imágenes contienen elementos extraños, como puntos y líneas por fuera del carácter principal, pero que podrían ser parte de la forma de escribir de ciertas personas (o por ejemplo en la i y la j, son parte del carácter).

Comencemos entonces descargando y revisando la documentación del dataset EMNIST (que contiene a MNIST): <https://www.kaggle.com/datasets/crawford/emnist/data>.

Si miramos la documentación, veremos que existe ya un subconjunto del dataset balanceado, para el cual tenemos 3000 ejemplos de cada clase. Véase que los creadores del dataset se dieron cuenta de algo muy particular, algunas letras mayúsculas no se diferencian de sus versiones minúsculas. Entonces han agrupado las clases de ambas en una sola. Siempre hay que leer la documentación del dataset para entender los datos con los que se está trabajando:



Laboratorio: Deep Learning Desde Cero con CNNs

Parte 2 - Modelo

Utilizaremos un modelo secuencial ([TF Keras Sequential Models](#)) que refiere a una red en la que se apilan capas una detrás de la otra, que es la forma clásica de crear redes profundas. ¿Qué capas podemos usar? Siempre que sean compatibles una capa con otra, podemos usar cualquiera de las capas definidas ([TF Keras Layers](#)), en particular nos interesarán las capas usuales en redes convolucionales:

- [Conv2D](#): opera el input con convoluciones (entrenables) en 2 dimensiones, esto es, los filtros son 2D,
- [MaxPool2D](#): pooling baja el tamaño del input tomando el mayor valor en una ventana dada (no entrenable),
- [Flatten](#): convierte un input en un arreglo contiguo (no entrenable), útil para conectarlo a una capa densa, cuando la información ya está “resumida”,
- [Dense](#): capa de conexiones densa (entrenable), es decir, conecta todas las entradas con todas las salidas.

Crearemos una estructura de red secuencial con la siguiente arquitectura. Debería ser sencillo siguiendo los links que se mencionaron en esta sección.

- Input 28x28x1
- Conv2D 32x3x3
- MaxPooling 2x2
- Conv2D 64x3x3
- MaxPooling 2x2
- Flatten
- Dense 128
- Dense (N_CLASSES)

En general armar una red es un proceso iterativo y un tanto creativo. Uno comienza por el tamaño de las entradas, por ejemplo en nuestro caso un “tensor” (matriz de más de 2 dimensiones) de 28x28x1 píxeles (el último son los canales de color, nosotros tenemos uno solo, nivel de blanco/negro), y luego decide qué espera que la red aprenda.

Por ejemplo suponemos que la red aprenderá a reconocer pequeños trazos, entonces elegimos una cantidad arbitraria de convoluciones pequeñas, como en nuestro caso 32 convoluciones de 3x3 píxeles. Si hacemos los cálculos esto nos lleva a una serie de 32 matrices de 26x26 (o un tensor de 26x26x32). Reducimos su tamaño con un max-pooling de 2x2, lo que nos lleva a 32 matrices de 13x13 y continuamos de la misma forma.

Al final, como queremos llegar a un vector de probabilidades para las distintas clases, siempre aplanaremos la salida a un vector unidimensional con Flatten. Luego realizamos un par de capas densas para que la red pueda tomar todos estos “features” que pre-procesamos y los clasifique en una de las N clases que queremos clasificar.

Laboratorio: Deep Learning Desde Cero con CNNs

El resultado final de la red es una serie de números, que dependen de la función de activación que utilizamos en la última capa:

- Sigmoid: es una función que toma valores entre 0 y 1, por lo que nos devolverá un valor de “verosimilitud” de que nuestra entrada pertenezca a la clase correspondiente. Si este valor lo procesamos con una función como Softmax obtendremos así “probabilidades” de que la entrada pertenezca a la clase correspondiente.
- Tangente Hiperbólica: devuelve valores entre -1 y 1, que puede prevenir que se desvanezca el gradiente en ciertos casos.
- Linear: función que devuelve valores en el rango de los reales, restringido por el tipo de datos. Se usa para tareas de regresión, donde queremos predecir un valor y no una clase.

Parte 3 - Métricas y Optimizadores

Métricas

Necesitamos una fórmula que, dado un vector de valores entre 0 y 1 (para pensar: ¿puede ser 0 o 1? ¿de qué depende?), representando la “confianza” del modelo en que un ejemplo pertenece a una clase dada, y el valor real de dicha clase, nos devuelva una métrica de error o “loss” para luego propagar hacia atrás y corregir los pesos de la red.

Si tuviésemos un clasificador binario, en el cual nuestra clase toma los valores 0 o 1 y la salida de la red es un valor entre 0 y 1, podríamos usar la “Binary Cross-Entropy”:

$$\text{Loss} = -\frac{1}{\text{output size}} \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i + (1 - y_i) \cdot \log (1 - \hat{y}_i)$$

En nuestro caso tendremos una única clase correcta (e.g. [0, 0, 1, 0]) y una serie de “probabilidades” predichas para cada clase (e.g. [0.1, 0.2, 0.5, 0.2]), en este caso podemos calcular el error con “Categorical Cross-Entropy”:

$$\text{Loss} = -\sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i$$

(para pensar: ¿qué estamos penalizando acá y cómo?)

Otras posibles métricas implementadas en Tensorflow para éste tipo de problemas son:

Laboratorio: Deep Learning Desde Cero con CNNs

- CategoricalFocalCrossentropy
- CategoricalHinge

Optimizadores

Hasta ahora vimos que el entrenamiento de redes funciona gracias al descenso por el gradiente estocástico (SGD por las siglas de “Stochastic Gradient Descent”). En términos básicos el SGD funciona de la siguiente forma, dado un peso w y un gradiente (ya calculado) g , aplica una corrección pesada por el learning rate:

$$w = w - \text{learning_rate} * g$$

en caso de haber activado el momentum, la corrección incorpora un factor v de la dirección de corrección previa:

$$w = w + v * \Delta w_{-1} - \text{learning_rate} * g$$

Sin embargo, estos optimizadores se usan para muchos modelos distintos, y como es obvio, distintos usuarios notaron problemas en el descenso estocástico (como ruido, o falta de adaptabilidad), y desarrollaron otras fórmulas ([TF Keras Optimizers](#), [Wikipedia: Stochastic Gradient Descent - Extensions and Variants](#)) que intentan resolver algunos de éstos problemas.

En particular utilizaremos Adam (de “Adaptive Moment Estimation”), que además de incorporar momentum, agrega un promedio de los últimos pasos dados (como RMSProp y AdaGrad), cuya influencia disminuye con el tiempo, y agrega un término en base a los segundos momentos de los mismos (la segunda derivada).

Para un tratamiento extensivo de esta sección y la próxima recomendamos leer:

<https://cs231n.github.io/neural-networks-3/#update>

Parte 4 - Entrenamiento

El entrenamiento es un proceso sencillo, lo único que necesitamos es elegir un número adecuado de épocas, dar un tamaño de batch (ya sea por disponibilidad de memoria o para mejorar la convergencia), e incorporar un conjunto de validación.

Tensorflow permite además pasarle al entrenamiento funciones callback¹, que se llaman durante las distintas etapas del entrenamiento. Algunas de las funciones usuales son:

¹ https://www.tensorflow.org/guide/keras/writing_your_own_callbacks#an_overview_of_callback_methods

Laboratorio: Deep Learning Desde Cero con CNNs

- **ModelCheckpoint**: guarda el modelo o los pesos de la red con cierta cadencia, o cuando se cumplen ciertas condiciones.
- **EarlyStopping**: evita continuar entrenando cuando alguna métrica deja de mejorar, es decir, cuando se llega a una meseta.
- **LearningRateScheduler** y **ReduceLROnPlateau**: permiten modificar el learning-rate bajo ciertas condiciones ; el segundo en particular lo reduce cuando alguna métrica (usualmente la validación) no ha mejorado por cierto tiempo.
- **TensorBoard**: envía información a una interfaz gráfica que permite monitorear todo el entrenamiento y plotear distintas métricas, incluso durante el entrenamiento.

En principio le recomendamos utilizar schedulers de learning-rate, junto con un subconjunto de validación apropiado. Luego puede incorporar otros elementos, pero estos dos ya se consideran básicos.

Además, la función de entrenamiento de Tensorflow, cuando termina de entrenar, devuelve una variable con información sobre el entrenamiento. Podemos inspeccionarla y utilizarla para graficar algunos valores del entrenamiento (elegidos y seteados previamente).

Parte 5 - Evaluación

Con la red ya entrenada y seleccionada, podemos pasar a mostrar y medir el error.

¡Recuerde! Una vez utilizado un “conjunto de test” para medir o comparar la performance de distintas redes, no se puede usar más. Si empieza a buscar la red que mejore el rendimiento en ese conjunto de test estará haciendo overfitting sobre el mismo.

Dependiendo si se trabaja con un problema de regresión, clasificación binaria o multi-clase, o inclusive si la importancia relativa entre clasificar bien o mal las distintas clases es diferente, se buscarán distintas métricas² que permitan comprender si el problema está bien resuelto o no. Si les interesa profundizar en la teoría de cómo elegir una métrica que represente correctamente el error, les recomiendo leer las publicaciones o videos de Luciana Ferrer (UBA-CONICET)³ sobre “Proper Scoring Rules”.

En nuestro caso terminamos con una red que nos devuelve N valores, correspondientes cada uno a una clase, que representan una probabilidad de que nuestra entrada corresponda a cada una de las clases. Si quisiéramos aplicar métricas de clasificación tendríamos que primero quedarnos con la clase de mayor probabilidad⁴, y luego computar algo como el zero-one loss.

Pero la red nos dio probabilidades, entonces podemos hacer algo mejor, podemos además quedarnos con la probabilidad de esta clase y sólo considerarla positiva si cae por sobre algún valor de corte (e.g. más de 0.5 o

² https://scikit-learn.org/stable/modules/model_evaluation.html#classification-metrics

³ <https://www.liaa.dc.uba.ar/equipo/luciana-ferrer/>, o buscar “Evaluación responsable de sistemas de IA” por la Fundación Vía Libre, disponible en YouTube.

⁴ Por ejemplo usando `np.argmax`

Laboratorio: Deep Learning Desde Cero con CNNs

50%). O incluso podríamos revisar si la clase correcta se encuentra en el top-n de clases, o si hay una diferencia importante entre la de mayor probabilidad y las siguientes, etc.

En nuestro caso supondremos que queremos el caracter de mayor probabilidad siempre, entonces calcularemos el Accuracy, y visualizaremos la matriz de confusión para ver cuándo se equivoca nuestra red.

Conclusiones, Discusión y Trabajo Futuro

Les proponemos pensar al menos una pregunta para hacerse en cada parte. Párense en cada sección y digan: ¿qué más podría querer saber sobre esta sección?. Escriban las preguntas y las respuestas.

Además nos interesaría que prueben cambiando alguna parte del proceso. Busquen alguna otra forma de regularización, otro optimizador, u otra arquitectura de red. ¿Qué resultados obtienen? ¿Por qué creen que se dan estos resultados? (hagan una suposición inteligente, no busquen probarlo formalmente).

Finalmente analizen los resultados obtenidos. ¿Son satisfactorios? ¿Funcionarían en la vida real? ¿Qué le cambiarían o agregarían para hacerlo útil?

Por su cuenta

Finalmente llegamos a un detector que puede reconocer y clasificar correctamente caracteres manuscritos (ya sean números o letras). ¿Cómo pasamos a leer un documento manuscrito entero?

Les dejamos como trabajo propio (fuera del programa de la materia) implementar un método conocido como [YOLO \(You Only Look Once\)](#), que funciona como un “detector de objetos”, en el cual nuestros objetos serán caracteres manuscritos sobre una hoja o documento.

Brevemente lo que hacen los autores de YOLO es tomar una red convolucional entrenada para clasificación de objetos en ImageNetv3, congelan la mayor parte de los pesos aprendidos por la red, y luego le agregan alguna serie de capas al final que entrenan con otro dataset más pequeño para detectar elementos (coordenadas de rectángulos que encierran a los objetos). Un proceso conocido como “Transfer Learning”.

En nuestro caso, ya tenemos una red entrenada para clasificar caracteres, podemos aprovecharla para generar un detector de caracteres. Deberían encontrar un dataset anotado (o anotarlos ustedes mismos) con los “bounding box” de caracteres manuscritos en un documento, que les permita aprovechar la representación aprendida por nuestra red convolucional.

*CONSEJO: el dataset EMNIST les da caracteres sueltos. ¿Cómo pueden usarlos para **generar** datos con bounding boxes automáticamente anotados para entrenar YOLO a detectar caracteres?*